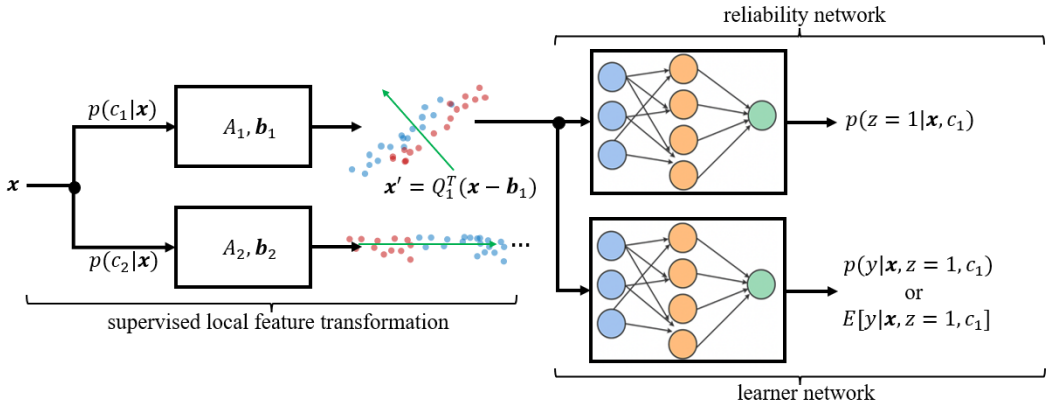


Figure 2: Overall architecture of our framework.

3.1 Basic Structure and Properties

Figure 2 illustrates the overall architecture of our framework, which, according to Equation 1, seeks to model variation in predictability across the feature space. In addition to the neural network used for classification or regression (the learner), we employ a separate neural network to learn the reliability score $w(\boldsymbol{x}) = p(z = 1 \mid \boldsymbol{x})$. Throughout this work, we use MLPs with a single hidden layer of 64 neurons for both networks, leaving the exploration of more complex architectures for future work. We also recognize that different regions of the feature space may benefit from more carefully tuned local models, particularly in terms of feature transformation, the extension will be discussed in Section 3.3.

For a neural classifier, the conditional distributions of the response, $p(y \mid \boldsymbol{x}, z = 1)$ and $p_{\text{rand}}(y)$, are simply class probabilities. For a neural regressor, we further model these conditional distributions as Gaussian, i.e.,

$$y \mid \boldsymbol{x}, z = 1 \sim N\left(y; f(\boldsymbol{x}), \sigma_1^2\right), \; y \mid \boldsymbol{x}, z = 0 \sim N\left(y; \mu_0, \sigma_0^2\right), \tag{2}$$

where f, μ_0, σ_0^2 and σ_1^2 are unknown functions and parameters. In other words, when $z = 1$, we have $\mathbb{E}[y \mid \boldsymbol{x}, z = 1] = f(\boldsymbol{x})$, whereas when $z = 0$, y is independent of $\boldsymbol{x}$ and behaves like white noise.

Notice that in both cases, we have

$$\begin{aligned} p(y, z \mid \boldsymbol{x}) &= p(z \mid \boldsymbol{x}) p(y \mid \boldsymbol{x}, z) \\ &= p(z = 1 \mid \boldsymbol{x})^z [1 - p(z = 1 \mid \boldsymbol{x})]^{1-z} p(y \mid \boldsymbol{x}, z = 1)^z p_{\text{rand}}(y)^{1-z}, \end{aligned} \tag{3}$$

hence, the joint log-density of y and z can be obtained as

$$\begin{aligned} \log p(y, z \mid \boldsymbol{x}) = & \underbrace{z \log p(z = 1 \mid \boldsymbol{x}) + (1 - z) \log[1 - p(z = 1 \mid \boldsymbol{x})]}_{\text{reliability network}} \\ & + \underbrace{z \log p(y \mid \boldsymbol{x}, z = 1)}_{\text{learner network}} + \underbrace{(1 - z) \log p_{\text{rand}}(y)}_{\text{random-response component}}, \end{aligned} \tag{4}$$

which is naturally decomposed into three components corresponding to the reliability network, the learner network, and the estimation of the distribution of the unpredictable component, respectively.

Ideally, according to Equation 4, if the latent variable z is known, the learner network should be trained on instances with $z = 1$, which should lead to it learn the distribution of the predictable component $p(y \mid \boldsymbol{x}, z = 1)$. However, since z is not observable, as we will demonstrate in Section 3.2, the empirical learner target used in training is actually the log likelihood of the clean component weighted by the posterior reliability

$$\begin{aligned} \gamma(\boldsymbol{x}, y) :&= p(z = 1 \mid \boldsymbol{x}, y) \\ &= \frac{p(z = 1 \mid \boldsymbol{x}) p(y \mid \boldsymbol{x}, z = 1)}{p(z = 1 \mid \boldsymbol{x}) p(y \mid \boldsymbol{x}, z = 1) + p_{\text{rand}}(y)[1 - p(z = 1 \mid \boldsymbol{x})]}. \end{aligned} \tag{5}$$

The good thing is, assuming that training converges to the optimum, this procedure also leads the learner to recover the predictable component of the response, which is precisely the distribution the learner is intended to model. See Appendix A for the proofs of the following results.